C++编程语言中赋值运算符重载函数（operator=）介绍

原创 liitdar 已于 2024-05-30 11:57:26 修改 阅读量10w+ 收藏 567 点赞数 149 版权

分类专栏： C/C++编程语言 文章标签： C++ operator 重载赋值运算符

本文主要介绍C++编程语言中赋值运算符重载函数（operator=）的相关知识，同时通过示例代码介绍赋值运算符重载函数的使用方法。

1 概述

1.1 Why

首先介绍为什么要对赋值运算符“=”进行重载。某些情况下，当我们编写一个类的时候，并不需要为该类重载“=”运算符，因为编译系统为每个类提供了默认的赋值运算符“=”，使用这个默认的赋值运算符操作类对象时，该运算符会把这个类的所有数据成员都进行一次赋值操作。例如有如下类：

```
class A
{
public:
    int a;
    int b;
    int c;
};
```

对这个类的对象进行赋值时，使用默认的赋值运算符是没有问题的。

示例代码内容如下：

```
#include <iostream>

using namespace std;

class ClassA
{
public:
    int a;
```

```
    int b;
    int c;
};

int main()
{
    ClassA obj1;
    obj1.a = 1;
    obj1.b = 2;
    obj1.c = 3;

    ClassA obj2;
    obj2 = obj1;

    cout << "obj2.a is: " << obj2.a << endl;

    return 0;
}

```

编译并执行上述代码，结果如下：

```
[root@node1 /opt/liitdar/mydemos/simples]# ./operator_test4
obj2.a is: 1
[root@node1 /opt/liitdar/mydemos/simples]#
```

https://blog.csdn.net/liitdar

通过上述结果能够知道：通过使用默认的赋值运算符“=”，可以让对象obj2中的所有数据成员的值与对象obj1相同。这种情况下，编译系统提供的默认赋值运算符可以正常使用。

但是，在下面的示例中，使用编译系统提供的默认赋值运算符，就会出现问题了。

示例代码内容如下：

```
#include <iostream>
#include <string.h>

using namespace std;

```

```
class ClassA 7 {
public:
    ClassA()
    {

    }

    ClassA(const char* pszInputStr)
    {
        pszTestStr = new char[strlen(pszInputStr) + 1];
        strncpy(pszTestStr, pszInputStr, strlen(pszInputStr) + 1);
    }
    virtual ~ClassA()
    {
        delete pszTestStr;
    }
public:
    char* pszTestStr;
};

int main()
{
    ClassA obj1("liitdar");

    ClassA obj2;
    obj2 = obj1;

    cout << "obj2.pszTestStr is: " << obj2.pszTestStr << endl;
    cout << "addr(obj1.pszTestStr) is: " << &obj1.pszTestStr << endl;
    cout << "addr(obj2.pszTestStr) is: " << &obj2.pszTestStr << endl;

    return 0;
}
```

编译并运行上述代码，结果如下：

```
[root@node1 /opt/liitdar/mydemos/simples]# ./operator_test5
obj2.pszTestStr is: liitdar
addr(obj1.pszTestStr) is: 0x7fffe2648ba8
addr(obj2.pszTestStr) is: 0x7fffe2648b98
*** Error in `./operator_test5': double free or corruption (fasttop): 0x00000000006ca010 ***
======= Backtrace: =========
/lib64/libc.so.6(+0x7c619)[0x7f78ca522619]
```

https://blog.csdn.net/liitdar

上述错误信息表明：当对象obj1和obj2进行析构时，由于重复释放了同一块内存空间，导致程序崩溃报错。在这种情况下，就需要我们重载赋值运算符“=”了。

2 示例代码

2.1 示例代码1

我们修改一下上面出错的示例代码，编写一个包含 赋值运算符重载 函数的类，修改后的代码内容如下：

```
#include <iostream>
#include <string.h>

using namespace std;

class ClassA
{
public:
    ClassA()
    {

    }
    ClassA(const char* pszInputStr)
    {
        pszTestStr = new char[strlen(pszInputStr) + 1];
        strncpy(pszTestStr, pszInputStr, strlen(pszInputStr) + 1);
    }
    virtual ~ClassA()
    {
        delete pszTestStr;
    }
    // 赋值运算符重载函数
    ClassA& operator=(const ClassA& cls)
```

```
    {
        // 避免自赋值
        if (this != &cls)
        {
            // 避免内存泄露
            if (pszTestStr != NULL)
            {
                delete pszTestStr;
                pszTestStr = NULL;
            }

            pszTestStr = new char[strlen(cls.pszTestStr) + 1];
            strncpy(pszTestStr, cls.pszTestStr, strlen(cls.pszTestStr) + 1);
        }

        return *this;
    }

public:
    char* pszTestStr;
};

int main()
{
    ClassA obj1("liitdar");

    ClassA obj2;
    obj2 = obj1;

    cout << "obj2.pszTestStr is: " << obj2.pszTestStr << endl;
    cout << "addr(obj1.pszTestStr) is: " << &obj1.pszTestStr << endl;
    cout << "addr(obj2.pszTestStr) is: " << &obj2.pszTestStr << endl;

    return 0;
}
```

编译并运行上述代码，结果如下：

```
[root@node1 /opt/liitdar/mydemos/simples]# ./operator_test5
obj2.pszTestStr is: liitdar
addr(obj1.pszTestStr) is: 0x7ffdb7702da8
addr(obj2.pszTestStr) is: 0x7ffdb7702d98
[root@node1 /opt/liitdar/mydemos/simples]#
```

http://blog.csdn.net/liitdar

通过上述结果能够看到，利用赋值运算符重载函数，解决了对象赋值时，析构函数多次释放同一块内存空间的问题。

对于上述代码，有以下几点需要说明：

- 当为一个类的对象赋值（可以用本类对象为其赋值，也可以用其它类型的值为其赋值）时，该对象（如本例的obj2）会调用该类的赋值运算符重载函数，进行具体的赋值操作。如上述代码中的“obj2 = obj1;”语句，用obj1为obj2赋值，obj2会调用ClassA类的赋值运算符重载函数；
- 下方语句和语句“ClassA obj2 = obj1;”在调用函数时是有区别的：前者第一句是对象obj2的声明及定义，调用类ClassA的无参构造函数，所以“obj2 = obj1;”一句是在对象obj2已经存在的情况下，用obj1来为obj2赋值，调用的是赋值运算符重载函数；而后者，是用obj1来初始化obj2，调用的是拷贝构造函数。拷贝构造函数的语句样式为“ClassA(const ClassA& cls)”。关于拷贝构造函数的详细知识，请查询相关内容，此处不展开介绍；

```
ClassA obj2;
obj2 = obj1;
```

- 当程序没有显式地提供一个以“本类或本类的引用”为参数的赋值运算符重载函数时，编译器会自动生成一个默认的赋值运算符重载函数（即默认赋值运算符）。

2.2 示例代码2

示例代码2内容如下：

```
#include<iostream>
#include<string>

using namespace std;

class Data
{
private:
    int data;
```

```
10
11 public:
12     // 构造函数
13     Data()
14     {
15     };
16     // 构造函数
17     Data(int _data):data(_data)
18     {
19         cout << "This is constructor" << endl;
20     }
21     // 赋值运算符重载函数
22     Data& operator=(const int _data)
23     {
24         cout << "This is operator=(int _data)" << endl;
25         data = _data;
26
27         return *this;
28     }
29 };
30
31 int main()
32 {
33     // 调用构造函数
34     Data data1(1);
35     Data data2, data3;
36     // 调用赋值运算符重载函数
37     data2 = 1;
38     // 调用默认的赋值运算符重载函数
39     data3 = data2;
40
41     return 0;
42 }
```

编译并执行上述代码，结果如下：

```
[root@node1 /opt/liitdar/mydemos/simples]# ./operator_test6
This is constructor
This is operator=(int _data)
[root@node1 /opt/liitdar/mydemos/simples]#
```

https://blog.csdn.net/liitdar

上述结果表明：“data2 = 1;”语句调用了我们提供的以int类型参数（而非本类或本类的引用）为形参的赋值运算符重载函数；而“data3 = data2;”的成功执行，说明该语句调用了编译器提供的默认的赋值运算符重载函数。

如果将上述代码中赋值运算符重载函数去掉，重新编译执行，结果如下：

```
[root@node1 /opt/liitdar/mydemos/simples]# ./operator_test6
This is constructor
This is constructor
[root@node1 /opt/liitdar/mydemos/simples]#
```

https://blog.csdn.net/liitdar

上述结果说明，当用一个非类“A”的值（如上面的int类型值）为类“A”的对象赋值时：

- 如果检测到构造函数和赋值运算符重载函数同时存在，则会优先调用赋值运算符重载函数；
- 如果只检测到构造函数，就会调用构造函数。

3 总结

综合本文内容，可以知道针对以下情况，需要**显式地**提供赋值运算符重载函数（即自定义赋值运算符重载函数）：

- 用非类“A”类型的值为类“A”的对象赋值时（当然，这种情况下我们也可以不提供相应的赋值运算符重载函数，而只提供相应的构造函数，如**更改后**的示例代码2）；
- 用类“A”类型的值为类“A”的对象赋值，且类“A”的数据成员中含有指针的情况下，必须显式提供赋值运算符重载函数（如示例代码1）。